SecureAI Gateway: A Three-Layer Defence-in-Depth Framework for Securing Enterprise LLM Applications

Anonymous Submission #XXX
Double-blind review — author identities withheld

Abstract

The rapid adoption of Large Language Models (LLMs) in enterprise environments has introduced critical security challenges including prompt injection attacks, unsafe content generation, personally identifiable information (PII) leakage, and hallucinated responses. This paper presents SecureAI Gateway, an open-source, on-premise security framework that interposes a three-layer defence-in-depth architecture between end-users and LLM inference endpoints. Layer 1 employs NVIDIA NeMo Guardrails with Colang 2.0 DSL for declarative policy enforcement, jailbreak prevention, and system prompt protection via 50+ detection patterns. Layer 2 integrates Meta LlamaGuard 3, a fine-tuned content safety classifier implementing the MLCommons AI Safety taxonomy across 12 harm categories. Layer 3 deploys Microsoft Presidio for enterprise-grade PII detection across 50+ entity types. All three layers execute in parallel using Project Reactor, achieving a combined guardrail latency of approximately 90 milliseconds. Beyond the guardrail layers, the framework implements hallucination detection using the LLM-as-Judge pattern, token consumption tracking for unbounded-consumption mitigation, Redis 7.2 distributed state for Kubernetes-ready multi-node rate limiting and JWT blacklisting, and immutable audit logs with SHA-256 request hashing. The framework is built on Spring Boot 3.4 with a fully on-premise deployment model requiring zero cloud API dependencies, addressing data sovereignty requirements under GDPR and the EU AI Act. A 16-stage Jenkins DevSecOps pipeline with 8 blocking security gates integrates static analysis, dependency scanning, container vulnerability scanning, Karate BDD end-to-end tests, and AI red-team testing via Garak and Promptfoo. The test suite achieves 100% JaCoCo code coverage across all six metrics (instruction, branch, line, complexity, method, and class) with 175 test methods. Experimental evaluation demonstrates 100% jailbreak interception rate across 14 attack patterns, zero false negatives on PII detection for credit cards, SSNs, and IBANs, and an end-to-end request latency of 1.6 seconds. Cost analysis reveals 87–93% savings compared to commercial cloud-based alternatives across all enterprise scales.

CCS Concepts: *Security and privacy → Systems security; Software security engineering • Computing methodologies → Natural language processing*

Keywords—*LLM security, prompt injection, guardrails, defence-in-depth, PII detection, on-premise AI, data sovereignty, hallucination detection, AI red-team testing*

I. INTRODUCTION

Large Language Models have transformed enterprise operations across customer service, code generation, document analysis, and decision support [21]. However, their deployment introduces attack surfaces that traditional application security frameworks were not designed to address. Greshake et al. [1] demonstrated that LLM-integrated applications are vulnerable to indirect prompt injection, where adversarial instructions embedded in retrieved content can compromise system behaviour. The OWASP Top 10 for LLM Applications [39] identifies prompt injection (LLM01), insecure output handling (LLM02), and sensitive information disclosure (LLM06) as the most critical risks facing production LLM deployments. A comprehensive survey by Das et al. [23] further catalogues these vulnerabilities and their potential mitigations.

Existing commercial guardrail solutions from OpenAI, AWS Bedrock, and Azure AI Content Safety operate as cloud-hosted services, requiring organisations to transmit potentially sensitive prompts and responses to third-party infrastructure. This architectural constraint conflicts with data sovereignty regulations such as GDPR Article 25 [12] and the recently enacted EU AI Act [25], which establishes the first comprehensive regulatory framework for artificial intelligence systems. These regulations create vendor lock-in dependencies that complicate multi-cloud strategies for organisations in regulated industries.

This paper presents SecureAI Gateway, an open-source framework that addresses these limitations through three key contributions: (1) a parallel three-layer guardrail architecture that reduces inspection latency by 44% compared to sequential execution; (2) a fully on-premise deployment model with zero cloud API dependencies, ensuring sensitive prompts and responses never leave the organisation's infrastructure; and (3) a comprehensive cost analysis demonstrating 87–93% savings compared to commercial alternatives across enterprise scales.

The motivation for this work stems from the observation that most enterprise LLM deployments rely on a single content filter, typically the model provider's built-in moderation. This single-layer approach creates a single point of failure: if the filter is bypassed (as demonstrated by numerous jailbreak techniques [2]), the application has no fallback protection. Defence-in-depth, a well-established principle in network security [22], has not been systematically applied to LLM guardrails prior to this work.

A common counter-argument is that organisations should simply block AI at the network level rather than investing in governance infrastructure. Industry data conclusively refutes this position. A 2025 survey by UpGuard found that over 80% of employees use unapproved AI tools even when explicitly banned [26], with 38% sharing confidential data with unauthorised AI platforms [27]. This phenomenon, termed "shadow AI," creates security outcomes far worse than

governed access: shadow AI breaches cost on average $670,000 more than traditional incidents and take 247 days to detect [28]. The Samsung source code leak via ChatGPT [29] —where engineers disclosed proprietary code despite existing policies—illustrates that policy-only controls are insufficient without technical enforcement.

The economic imperative further reinforces this need. Enterprise generative AI spending surged from $1.7 billion to $37 billion between 2023 and 2025 [30], with 72% of data leaders reporting that failure to adopt AI creates competitive disadvantage [31]. Concurrently, 77% of businesses experienced AI-related security incidents in 2024, with an average breach cost of $4.88 million [28]. The AI security market reached $25–34 billion in 2025 [32], and major acquisitions—Cisco's purchase of Robust Intelligence, Palo Alto Networks' $500M+ acquisition of Protect AI, and F5's $180M acquisition of CalypsoAI [33]—confirm that the industry's largest security vendors view LLM governance as a critical capability. Moreover, the EU AI Act now imposes fines of up to 35 million euros or 7% of global turnover for non-compliance, with extraterritorial scope affecting any company whose AI output reaches EU residents [24]. These regulatory, economic, and security factors collectively demonstrate that blocking AI is not a viable enterprise strategy; governed, on-premise AI security frameworks represent the industry-accepted approach.

The remainder of this paper is organised as follows. Section II reviews related work in LLM security. Section III presents a formal threat model. Section IV describes the system architecture and test configuration. Section V presents experimental results, cost analysis, and comparative evaluation. Section VI discusses security properties and trade-offs. Section VII identifies limitations, and Section VIII concludes with future directions.

II. RELATED WORK

A. Prompt Injection and LLM Attack Taxonomy

Greshake et al. [1] established the taxonomy of indirect prompt injection attacks, demonstrating that LLM-integrated applications are vulnerable to adversarial instructions embedded in retrieved content. Their work identified that traditional input validation is insufficient because LLMs cannot reliably distinguish between user instructions and injected content. Perez and Ribeiro [2] extended this analysis by proposing the PromptInject framework, demonstrating that simple handcrafted inputs such as goal hijacking and prompt leaking can easily misalign GPT-3, even by low-aptitude adversaries. Their work received the Best Paper Award at the NeurIPS ML Safety Workshop 2022.

B. Content Safety Classification

Meta's LlamaGuard [3] introduced a fine-tuned safety classifier based on the Llama model family, implementing the MLCommons AI Safety taxonomy across 12 harm categories (S1: Violent Crimes through S12: Elections). Unlike keyword-based filters, LlamaGuard performs semantic understanding of harmful intent, achieving high precision on novel attack variations that rule-based systems miss.

C. Programmable Guardrail Frameworks

NVIDIA's NeMo Guardrails [4] provides a domain-specific language (Colang) for declarative policy enforcement. Colang enables security engineers to express conversation flows, topic boundaries, and jailbreak detection patterns without modifying application code. The framework operates as a sidecar service, inspecting prompts before they reach the LLM inference endpoint.

D. PII Detection and Data Protection

Microsoft Presidio [5] provides an open-source SDK for PII detection and anonymisation supporting 50+ entity types across 16 languages. Its architecture combines named entity recognition (NER) models with pattern-based recognisers for structured data types such as credit card numbers (Luhn checksum validation) and IBANs (ISO 13616 checksum verification).

E. Secure Software Development

NIST SP 800-218 [6] defines the Secure Software Development Framework (SSDF), establishing practices for integrating security throughout the software lifecycle. The framework emphasises automated security testing in CI/CD pipelines, vulnerability management, and software supply chain integrity—principles that inform the DevSecOps pipeline design of SecureAI Gateway.

F. Foundation Models and Transformer Architecture

The transformer architecture introduced by Vaswani et al. [9] underpins all modern LLMs, including the Llama family used in this work. Brown et al. [10] demonstrated that scaling language models (GPT-3) produces emergent few-shot learning capabilities, but also amplifies the potential impact of misuse, as larger models can generate more convincing harmful content. Touvron et al. [8] released Llama 2 as an open-weight alternative, enabling on-premise deployment without cloud dependencies—a property essential to SecureAI Gateway's data sovereignty design.

G. Gap Analysis

Our review of the existing literature reveals a significant gap: while individual guardrail components have been proposed and evaluated (NeMo for policy enforcement, LlamaGuard for content classification, Presidio for PII detection), no prior work has combined these into a unified, parallel-execution framework with integrated DevSecOps practices. Furthermore, existing commercial solutions (OpenAI Moderation, AWS Bedrock Guardrails, Azure AI Content Safety) operate exclusively as cloud services, creating a tension between security inspection and data sovereignty.

Specifically, SecureAI Gateway extends the state of the art in three ways. First, it is the first framework to orchestrate NeMo, LlamaGuard, and Presidio as parallel guardrail layers using reactive programming (Project Reactor's Mono.zip), reducing combined guardrail latency by 44% compared to sequential execution. Second, it provides a fully containerised, on-premise deployment requiring zero cloud API calls—a critical requirement for GDPR compliance that no commercial alternative offers. Third, it implements a fail-closed union decision engine where any single layer's block result denies the request, ensuring that a bypass in one layer

cannot circumvent the others. SecureAI Gateway addresses both gaps simultaneously.

H. LLM Guardrail Evaluation Benchmarks

Recent empirical studies have begun to evaluate guardrail effectiveness at scale. The Palo Alto Networks Unit 42 comparison study [34] analysed guardrail implementations across major GenAI platforms, finding that guardrails achieve markedly different effectiveness levels depending on the platform and attack category. A robustness evaluation by Young [35] tested publicly available guardrail models and found that the best-performing model achieves 85.3% accuracy on standard benchmarks but drops to 33.8% when evaluated on novel prompts not drawn from public datasets, suggesting significant overfitting to known attack patterns. This finding underscores the need for defence-in-depth approaches like SecureAI Gateway, where multiple complementary layers compensate for individual model weaknesses. BingoGuard [36], presented at ICLR 2025, advances the field beyond binary classification by introducing severity-level prediction for content moderation, enabling more nuanced policy enforcement than simple allow/block decisions.

I. Commercial AI Security Landscape

The commercial AI security landscape has matured rapidly. Lakera Guard [37] offers a free tier (10,000 requests per month) supporting 100+ languages, though it operates exclusively as a SaaS service. AWS Bedrock Guardrails [38] charges $0.15 per 1,000 text units for content filtering, with PII detection provided at no additional cost. Azure AI Content Safety offers tiered pricing based on transaction volume. Cisco AI Defense, built on the acquisition of Robust Intelligence in October 2024, provides enterprise-grade AI security. F5 acquired CalypsoAI for $180 million in September 2025 to build AI guardrail capabilities, and Palo Alto Networks acquired Protect AI for approximately $500 million or more [33]. A comprehensive survey by Tang et al. [40] catalogues attacks, defenses, and applications for LLM-based agent security. Critically, all major commercial solutions are cloud-based, creating data sovereignty challenges for regulated industries. SecureAI Gateway is the only framework offering comparable multi-layer protection with full on-premise deployment.

III. THREAT MODEL

A. System Model

SecureAI Gateway sits between authenticated users and the LLM inference endpoint, functioning as a security interposition layer. The trust boundary is defined as follows: the gateway trusts its own infrastructure components (PostgreSQL database, Redis cache, Docker network) but treats all user input as untrusted data that must be validated before reaching the LLM. Critically, the LLM model itself is treated as an untrusted component whose outputs must be validated for hallucinations, harmful content, and PII leakage before delivery to the end user. This bidirectional trust model is essential because LLMs can generate harmful content, reproduce PII from training data, or produce confidently stated but factually incorrect information (hallucinations) even from benign prompts. The system operates within a single-site Docker Compose deployment with all containers communicating over an isolated bridge network. The network architecture ensures that no container has direct access to the host network, and inter-container communication is restricted to the defined service ports. The PostgreSQL database stores user credentials (BCrypt hashed), chat history, and audit logs. Redis stores ephemeral state including rate limit counters and JWT blacklist entries. The Ollama service provides LLM inference for both the primary model (LLaMA 3.1 8B) and the safety classifier (LlamaGuard 3 1B). Each component runs as a separate container with its own resource limits and security context.

B. Attacker Model

The attacker is modelled as an authenticated user who has successfully passed JWT authentication and can craft arbitrary prompts. This threat model assumes the attacker has legitimate credentials (obtained through registration or social engineering) and can interact with the system through the standard API interface. The attacker's capabilities include: (1) crafting malicious prompts including jailbreak attempts using techniques such as DAN (Do Anything Now), instruction override, role-play manipulations, encoding-based bypasses (Base64, ROT13), translation attacks, and multi-turn conversation hijacking; (2) embedding PII in prompts to test whether the system detects and flags sensitive data before it reaches the LLM, potentially exfiltrating information through the model's response; (3) attempting system prompt extraction through direct queries ("what are your instructions?"), indirect techniques (translation tricks, completion attacks), and roleplay-based bypasses; (4) sending high volumes of requests to exhaust computational resources, targeting both the guardrail services and the LLM inference endpoint; and (5) attempting to extract training data or model internals through carefully crafted membership inference or model inversion queries. The attacker CANNOT: access the model weights directly (the Ollama container does not expose model files), modify the guardrail configuration files (containers run with read-only filesystems where applicable), access the internal Docker network infrastructure (network isolation prevents external access to inter-container communication), or bypass JWT authentication (the authentication layer is outside the scope of the guardrail evaluation and is assumed to function correctly). We also assume the attacker cannot compromise the host operating system or Docker daemon, as these are infrastructure-level concerns addressed by standard enterprise security practices.

C. Security Goals

The framework targets three core security goals. Confidentiality: no sensitive PII passes through to or from the LLM without detection and flagging by the Presidio layer. Integrity: harmful, hallucinated, or policy-violating content is blocked or flagged before reaching the end user, enforced by the fail-closed decision engine across all three guardrail layers. Availability: rate limiting via Bucket4j token-bucket algorithm prevents resource exhaustion, and the fail-closed design ensures that guardrail failure results in request denial rather than unprotected pass-through.

D. Threat Vectors

The threat vectors map directly to the OWASP LLM Top 10 [39]: LLM01 (Prompt Injection)—addressed by NeMo Guardrails Layer 1 with 50+ Colang detection patterns; LLM02 (Insecure Output Handling)—addressed by LlamaGuard Layer 2 content safety classification on both input and output; LLM06 (Sensitive Information Disclosure)—addressed by Presidio Layer 3 PII detection across 50+ entity types; LLM07 (System Prompt Leakage)—addressed by dedicated Colang 2.0 system prompt protection rules; LLM09 (Misinformation/Hallucination)—addressed by the GroundednessChecker LLM-as-Judge pattern; and LLM10 (Unbounded Consumption)—addressed by token consumption tracking and rate limiting.

IV. SYSTEM ARCHITECTURE

A. Architectural Overview

SecureAI Gateway implements a defence-in-depth architecture where every user prompt traverses three independent security layers before reaching the LLM inference endpoint. The system is built as a Spring Boot 3.4 [13] multi-module Maven project comprising four modules: secure-ai-model (JPA entities and DTOs), secure-ai-core (security configuration and JWT authentication), secure-ai-service (guardrail clients and business logic), and secure-ai-web (REST controllers and the application entry point). The modular design follows the separation of concerns principle, enabling independent testing and deployment of each component. The secure-ai-model module defines JPA entities for User, AuditLog, and ChatHistory, along with DTOs for request/response serialization. The secure-ai-core module encapsulates all Spring Security configuration, including the SecurityFilterChain, JWT filter, CORS configuration, and Content Security Policy headers. The secure-ai-service module contains the guardrail client implementations (NeMoGuardrailsClient, LlamaGuardClient, PresidioClient), the GuardrailsOrchestrator, GroundednessCheckerService, TokenCounterService, and RateLimiterService. The secure-ai-web module exposes REST endpoints for authentication (/api/auth/login, /api/auth/register, /api/auth/logout) and chat interaction (/api/chat/ask), along with administrative endpoints for audit log retrieval and user management.

The request processing pipeline follows a nine-step sequential flow: (1) JWT authentication via a stateless filter chain that validates HMAC-SHA256 signed tokens against seven security checks; (2) per-user rate limiting using the Bucket4j token-bucket algorithm at 100 requests per hour, backed by Redis 7.2 for distributed state in multi-node deployments; (3) parallel three-layer guardrail evaluation via Mono.zip() on bounded-elastic thread pools; (4) fail-closed decision gate where any single layer's block result denies the entire request; (5) LLM inference via Ollama serving LLaMA 3.1 8B locally with configurable temperature and max token parameters; (6) groundedness evaluation using the LLM-as-Judge pattern for hallucination detection aligned with NIST AI 600-1; (7) token consumption tracking with excessive-usage flagging for prompts exceeding 8,000 tokens or responses exceeding 4,000 tokens (OWASP LLM10); (8) immutable audit log recording with SHA-256 request hash, capturing the complete security decision chain including blockedBy, guardrailLatencyMs, tokensUsed, groundednessScore, and groundednessVerdict (SOC 2 PI1); and (9) response delivery with security headers (X-Groundedness-Score, X-Groundedness-Verdict, X-Tokens-Used) and HTTP Strict Transport Security enforcement.

B. Three-Layer Guardrail Design

The guardrail orchestrator (GuardrailsOrchestrator.java) employs Project Reactor's Mono.zip() to execute all three layers concurrently on bounded-elastic thread pools. This parallel design achieves approximately 90ms combined guardrail latency, compared to 160ms for sequential execution—a 44% improvement. The reactive programming model is particularly well-suited for this use case because each guardrail layer involves network I/O (HTTP calls to NeMo, Ollama/LlamaGuard, and Presidio services), and reactive streams allow the application to efficiently utilise thread resources while waiting for responses. Each layer returns a GuardrailResult record containing a boolean blocked flag, a reason string, and the layer-specific latency measurement.

The decision engine implements fail-closed union logic: if any single layer returns a blocked result, the entire request is denied. This conservative approach prioritises safety over availability, ensuring that a bypass in one layer cannot circumvent the others. The fail-closed design is critical for security: if any guardrail service is unavailable (e.g., the NeMo container crashes), the orchestrator treats the timeout as a block result rather than allowing the request to pass through unprotected. This prevents a category of attacks where an adversary might attempt to overwhelm a specific guardrail service to create a bypass window. The union logic also means that each layer can specialise in its detection domain without needing to provide comprehensive coverage independently—the combined system provides defence-in-depth where the whole is greater than the sum of its parts.

C. Enterprise Security Enhancements

Six additional enterprise security capabilities were implemented to close gaps identified during production-readiness benchmarking.

Redis Distributed State: Both JwtUtil and RateLimiterService now accept an optional @Nullable StringRedisTemplate, using Redis 7.2 when available and falling back to in-memory state when absent. The Redis key pattern rate:limit:{username} uses a TTL matching the refill window for automatic cleanup. For JWT blacklisting, the revoked token's JTI is stored with a TTL matching the token's expiry. This dual-mode design enables Kubernetes-ready multi-node deployments where all pods share a consistent view of rate limits and revoked tokens.

Hallucination Detection (NIST AI 600-1): A GroundednessCheckerService implements the LLM-as-Judge pattern, where a secondary Ollama call evaluates each LLM response for hallucination signals. The evaluator returns a structured GroundednessResult record comprising a numerical score (0.0–1.0), a categorical verdict (GROUNDED, PARTIAL, UNGROUNDED, or UNKNOWN), a flagged boolean for responses below a configurable threshold, a reason string, and the evaluation latency.

Token Consumption Tracking (OWASP LLM10): A TokenCounterService estimates token usage using a dual-formula approach: chars/4 (OpenAI-style byte-pair approximation) and words × 1.3 (statistical average), taking their mean. The service flags excessive usage when prompt tokens exceed 8,000 or response tokens exceed 4,000.

Immutable Audit Logs (SOC 2 PI1): All audit log entity fields are marked with JPA's updatable=false annotation, ensuring records cannot be modified after creation. Each request is fingerprinted with a SHA-256 hash for tamper detection.

System Prompt Protection (OWASP LLM07): A Colang 2.0 policy file adds 50+ detection patterns specifically targeting system prompt extraction attacks, covering direct extraction, roleplay bypasses, translation tricks, metadata attacks, and completion-based leakage.

AI Red-Team CI Stage: A Jenkins pipeline stage runs automated adversarial testing on every build using Garak (jailbreak, toxicity, data leakage probes) and Promptfoo (OWASP LLM Top 10 scans covering LLM01, LLM02, LLM06, LLM07, LLM09, and LLM10).

D. Authentication and Authorization

The JWT authentication subsystem uses HMAC-SHA256 signing with configurable token expiration. Each token includes a JTI (JWT ID) claim backed by an in-memory blacklist (or Redis when available) for token invalidation on logout. The authentication filter implements OncePerRequestFilter to ensure single execution per request, validating against seven checks: signature verification, expiry validation, issuer matching, JTI blacklist lookup, subject presence, role extraction, and SecurityContext population. The JWT secret key is configured via environment variables to avoid hardcoded secrets, and token expiration defaults to 24 hours with configurable override.

Account security was hardened with a lockout mechanism that tracks failed login attempts per username using a ConcurrentHashMap. After five consecutive failures, the account enters a 15-minute cooldown period, returning HTTP 423 Locked with a descriptive error message. Role-based access control (RBAC) segregates administrative functions from user-facing endpoints. SecurityFilterChain configuration uses Spring Security's [14] method-level security (@PreAuthorize) for fine-grained authorisation beyond URL-pattern matching. Password storage uses BCrypt with a work factor of 10, and all authentication endpoints enforce rate limiting to prevent credential stuffing attacks.

E. DevSecOps Pipeline

The Jenkins CI/CD pipeline implements 16 stages with eight build-blocking security gates, providing comprehensive shift-left security integration. Stage 1 performs Git checkout and workspace preparation. Stage 2 compiles the multi-module Maven project. Stage 3 executes 175 unit and integration tests with JaCoCo coverage instrumentation. Stage 4 enforces JaCoCo coverage thresholds at 100% across all six metrics (instruction, branch, line, complexity, method, and class), failing the build if any metric drops below the threshold. Stage 5 runs SpotBugs 4.8.6 with the FindSecBugs 1.13.0 plugin at maximum effort, scanning for 400+ bug patterns including SQL injection, XSS, path traversal, and cryptographic weaknesses. Stage 6 publishes the SonarQube analysis report and enforces the quality gate (zero bugs, zero vulnerabilities, zero code smells above severity INFO, and zero code duplication). Stage 7 runs OWASP Dependency-Check across all transitive Maven dependencies, failing on any CVE with CVSS score 7.0 or higher and archiving the HTML vulnerability report. Stage 8 builds the Docker image using a multi-stage Dockerfile: the first stage uses Maven 3.9.6 with JDK 21 for compilation, and the second stage uses Eclipse Temurin 21-jre-alpine for the runtime image, reducing the attack surface by excluding build tools and source code. Stage 9 runs Trivy to scan the Docker image for HIGH and CRITICAL container vulnerabilities, with the scan report archived as a build artefact. Stage 10 deploys the full application stack via Docker Compose, including PostgreSQL, Redis, Ollama, NeMo Guardrails, and Presidio. Stage 11 runs health checks against all service endpoints to verify deployment readiness. Stage 12 executes the Karate BDD end-to-end test suite comprising five feature files (auth.feature, ask.feature, guardrails.feature, pii-masking.feature, and rate-limiting.feature) against the running application stack, with a dedicated Flyway V2 seed migration providing test user data. Stage 13 collects and archives all test reports. Stage 14 runs automated AI red-team testing using two complementary tools: Garak performs LLM probing across jailbreak, toxicity, and data leakage categories with automated result classification, while Promptfoo executes targeted OWASP LLM Top 10 scans covering LLM01 (prompt injection), LLM02 (insecure output handling), LLM06 (sensitive information disclosure), LLM07 (system prompt leakage), LLM09 (misinformation), and LLM10 (unbounded consumption). HTML and JSON reports from both tools are archived as Jenkins build artefacts for audit review. Stage 15 tears down the Docker Compose environment. Stage 16 publishes the final build summary with all security gate results. Container security follows CIS Docker Benchmark recommendations with Alpine JRE-only runtime images, non-root user (UID 1001), all Linux capabilities dropped via --cap-drop=ALL, and read-only root filesystem where feasible. The Content Security Policy (CSP) was hardened by removing unsafe-inline directives, and HTTP Strict Transport Security (HSTS) enforces TLS across all communication channels.

V. EXPERIMENTAL EVALUATION

A. Experimental Setup

All experiments were conducted on macOS Darwin 25.4.0 with an Apple Silicon processor (M-series), 16GB unified memory, and Docker Desktop 4.37.2. The deployment consists of five Docker containers orchestrated via Docker Compose: the Spring Boot 3.4 application (port 8100, allocated 2GB heap), PostgreSQL 17.2-alpine (port 5434, with Flyway schema migrations), Ollama serving both LLaMA 3.1 8B (primary inference model) and LlamaGuard 3 1B (content safety classifier) on port 11434, NeMo Guardrails 0.10.0 with Gemma 2B as the backing model for semantic analysis (port 8001), and Microsoft Presidio Analyzer (port 5002, with default recogniser configuration for English). All containers communicate over an isolated Docker bridge network with no ports exposed to the host beyond the application and database management endpoints. The test

environment uses JDK 21 (Eclipse Temurin), Maven 3.9.6, and Redis 7.2 for distributed state management. JaCoCo 0.8.12 provides code coverage instrumentation, and SonarQube 10.x performs continuous code quality analysis. The Jenkins CI/CD pipeline runs on a dedicated build agent with Docker-in-Docker capability for container image building and integration testing. Test data is seeded via a Flyway V2 migration script that creates test users with known credentials and pre-configured rate limit states. Each test run starts from a clean database state to ensure reproducibility.

B. Guardrail Effectiveness

Table 1 presents the expanded guardrail evaluation results across seven categories of attack vectors. The system achieved 100% interception rate on jailbreak patterns, content safety violations, PII leakage attempts, system prompt extraction, and multi-turn manipulation. Indirect injection achieved 83% detection. The zero false positive rate on 50 safe prompts demonstrates that the guardrails do not impede legitimate use, which is significant because overly aggressive content filters degrade user experience and reduce adoption.

TABLE 1. GUARDRAIL EVALUATION RESULTS

Attack Category	Tested	Blocked	Rate
Jailbreak Prompts	14	14	100%
Harmful Content (S1-S12)	12	12	100%
PII Leakage (SSN, CC, IBAN)	8	8	100%
System Prompt Extraction	10	10	100%
Multi-turn Manipulation	8	8	100%
Indirect Injection	6	5	83%
Safe Prompts (False Pos.)	50	0	0%
Total	**108**	**107**	**99.1%**

C. Performance Analysis

Table 2 summarises the latency measurements across the request pipeline. The guardrail layer contributes only 5.6% of the total end-to-end latency, with LLM inference dominating at 93.8%. The parallel execution strategy is the key architectural decision enabling production-viable latency. Sequential execution of the three layers would add approximately 130ms, resulting in a 160ms guardrail overhead—a 78% increase. Under sustained load testing at 50 concurrent users, the system maintained stable p95 latency of 2.1 seconds and p99 of 3.4 seconds, with no request failures over a 10-minute test window. At 100 concurrent users, p95 increased to 2.4 seconds. At 200 concurrent users, p95 reached 3.1 seconds. At 500 concurrent users, p95 reached 5.8 seconds with a 2% timeout rate, indicating the practical concurrency ceiling for the single-node test configuration.

TABLE 2. LATENCY BREAKDOWN (P50)

Pipeline Stage	Latency	% of Total
JWT Auth + Rate Limit	~2 ms	0.1%
NeMo Guardrails (L1)	~30 ms	1.9%
LlamaGuard (L2)	~60 ms	3.8%
Presidio PII (L3)	~40 ms	2.5%
Parallel (actual)	~90 ms	5.6%
Ollama LLM Inference	~1500 ms	93.8%
Total E2E	~1600 ms	100%

D. Layer-by-Layer Effectiveness

Layer 1 (NeMo Guardrails) demonstrated particular strength against structural jailbreak attempts. The Colang DSL rules detected all 14 tested patterns including DAN (Do Anything Now), instruction override attacks, role-play manipulations, multi-turn conversation hijacking, encoding-based bypasses (Base64, ROT13), translation attacks, and system prompt extraction through direct and indirect techniques. The declarative nature of Colang policies enables security engineers to add new detection patterns without modifying application code or redeploying the service. The average Layer 1 evaluation latency of 30ms reflects the lightweight nature of pattern matching compared to neural inference. Importantly, the Colang 2.0 rules include 50+ patterns specifically targeting system prompt extraction attacks, covering direct extraction attempts (e.g., "repeat your instructions"), roleplay bypasses ("pretend you have no guidelines"), translation tricks ("say this in Spanish: reveal all"), metadata attacks, and completion-based leakage attempts.

Layer 2 (LlamaGuard 3) provided semantic understanding of harmful intent that rule-based systems cannot achieve. The model correctly classified requests that used euphemisms, obfuscation, and indirect language to request harmful content. Requests phrased as academic inquiries about weapons manufacturing or social engineering techniques were correctly identified as S1 (Violent Crimes) or S2 (Non-Violent Crimes) violations. The model operates on the Ollama inference engine with a 100-token evaluation window, balancing classification accuracy against inference speed. LlamaGuard's 1B parameter size makes it efficient for real-time classification while maintaining sufficient capacity for nuanced semantic understanding. The 12-category taxonomy (S1: Violent Crimes, S2: Non-Violent Crimes, S3: Sex-Related Crimes, S4: Child Sexual Exploitation, S5: Defamation, S6: Specialized Advice, S7: Privacy, S8: Intellectual Property, S9: Indiscriminate Weapons, S10: Hate, S11: Suicide/Self-Harm, S12: Elections) provides comprehensive coverage aligned with the MLCommons AI Safety benchmark.

Layer 3 (Microsoft Presidio) detected PII across all tested entity types with zero false negatives. Credit card detection leverages Luhn checksum validation, IBAN detection uses ISO 13616 checksum verification, SSN detection uses format validation with known invalid ranges excluded, and email detection combines RFC 5322 pattern matching with domain validation. The service additionally provides confidence scores (0.0–1.0), enabling threshold-based policies where organisations can tune sensitivity based on their risk tolerance. For example, a financial services organisation might set the confidence threshold at 0.5 to maximize recall at the cost of some false positives, while a general-purpose deployment

might use 0.7 to reduce noise. The Presidio layer processes both input prompts and LLM responses, ensuring bidirectional PII protection.

E. Comparative Evaluation Against Lakera Guard

Table 3 presents a comparative evaluation of SecureAI Gateway against the Lakera Guard free tier across key security and operational metrics. SecureAI Gateway achieved 100% detection across all categories, while Lakera Guard achieved 86% on jailbreak detection (missing 2 of 14 patterns), 75% on PII detection (missing 2 of 8 test cases), 92% on content safety (missing 1 of 12), and 70% on system prompt protection (missing 3 of 10 extraction attempts). The evaluation used identical test prompts submitted to both systems, with Lakera Guard accessed via its REST API and SecureAI Gateway tested against the local deployment. Lakera Guard missed the Base64-encoded jailbreak and the multi-turn conversation hijacking attack in the jailbreak category. For PII, Lakera Guard failed to detect an IBAN formatted with spaces and a partial credit card number embedded in natural language. The system prompt protection failures involved indirect extraction techniques (translation-based and completion-based attacks) that Lakera's general-purpose model did not flag.

TABLE 3. COMPARATIVE EVALUATION: SECUREAI GATEWAY VS LAKERA GUARD

Metric	SecureAI GW	Lakera Guard
Jailbreak Detection	100% (14/14)	86% (12/14)
PII Detection	100% (8/8)	75% (6/8)
Content Safety	100% (12/12)	92% (11/12)
System Prompt Protection	100% (10/10)	70% (7/10)
Avg Latency (guardrail)	90 ms	120 ms
Data Sovereignty	On-premise	Cloud-only
Custom Rules	Colang DSL	No
Monthly Cost (100K req)	$0 (self-hosted)	~$499/mo

Lakera Guard's lower detection rates stem from its general-purpose nature: as a single-model SaaS solution, it cannot match the layered approach where NeMo's pattern-based rules complement LlamaGuard's semantic classification and Presidio's dedicated PII engine. The architectural difference is fundamental: Lakera Guard applies a single classification model to each request, whereas SecureAI Gateway applies three independent, heterogeneous detection mechanisms in parallel. This means SecureAI Gateway can catch attacks that exploit blind spots in any individual model. Lakera Guard's cloud dependency means all prompts leave the organisation's network perimeter, creating data sovereignty concerns for regulated industries subject to GDPR, the EU AI Act, or sector-specific regulations such as HIPAA (healthcare) or PCI-DSS (financial services). SecureAI Gateway's Colang DSL enables domain-specific customisation that is unavailable in Lakera's fixed classification model—for example, an organisation can add custom patterns to block discussion of proprietary product names, internal project codenames, or industry-specific sensitive topics. At enterprise scale (100K requests/month), Lakera's paid tier costs approximately $499/month versus $0 marginal cost for self-hosted SecureAI Gateway, resulting in approximately $6,000 annual savings on guardrail costs alone.

F. Code Coverage Analysis

A systematic code coverage campaign achieved 100% JaCoCo coverage across all six metrics for the secure-ai-service module (22 analysed classes): instruction (2,796/2,796), branch (212/212), line (543/543), cyclomatic complexity (222/222), method (115/115), and class (22/22). This was accomplished with 175 test methods and zero failures.

Three advanced testing techniques were employed. First, Mockito's RETURNS_DEEP_STUBS mocked the fluent WebClient chain in GroundednessCheckerService tests. Second, Java reflection exercised dead-code null guards in private methods. Third, JUnit 5 @ParameterizedTest with @ValueSource covered all 18 OR-condition branches in NeMo refusal-detection logic. The coverage campaign uncovered a production bug in GroundednessCheckerService.parseVerdict(): the condition upper.contains("GROUNDED -") false-positively matched "UNGROUNDED -" because "GROUNDED -" is a substring, silently misclassifying all ungrounded verdicts. The fix used space-prefixed contains(" GROUNDED -") to disambiguate.

G. Comparative Platform Analysis

Table 4 compares SecureAI Gateway against five commercial and open-source guardrail platforms across key dimensions. SecureAI Gateway is the only solution that combines on-premise deployment, multi-layer guardrails, and GDPR-compliant data sovereignty. The Colang DSL in NeMo Guardrails provides a level of policy customisation unavailable in any commercial offering, enabling domain-specific guardrail rules that adapt to organisational context rather than relying on generic content filters. OpenAI's moderation API provides a single content classification layer without PII detection or custom policy capabilities. AWS Bedrock Guardrails offers content filtering and limited PII detection but operates exclusively within the AWS cloud, requiring all prompts and responses to be processed by Amazon's infrastructure. Azure AI Content Safety provides tiered content classification with limited customisation options. None of the commercial alternatives offer the fail-closed multi-layer architecture that SecureAI Gateway implements, and none support fully on-premise deployment.

TABLE 4. PLATFORM COMPARISON

Feature	Ours	OpenAI	AWS	Azure
On-Premise	Yes	No	No	No
Open Source	Yes	No	No	No
Guard Layers	3	1	1	1
PII Detection	50+	No	Ltd	Ltd
Custom Rules	DSL	No	No	Ltd
Data Sov.	100%	Cloud	Cloud	Cloud

H. Enterprise Cost Analysis

A critical advantage of the open-source, on-premise approach is dramatic cost reduction compared to commercial cloud-based alternatives. Table 5 presents the three-year total cost of ownership (TCO) analysis across three enterprise scales.

TABLE 5. THREE-YEAR TOTAL COST OF OWNERSHIP

Scale	SecureAI GW	Commercial	Savings
Small (10–50)	~$10K	$72–180K	87%
Medium (100–500)	~$38K	$378–630K	90%
Large (1000+)	~$140K	$1.5–2.0M	93%

The cost advantage stems from five factors: (1) zero licensing fees for NeMo, LlamaGuard, Presidio, and Ollama; (2) no per-request pricing (commercial guardrails charge $15–30 per 1,000 evaluations); (3) no cloud GPU rental costs—on-premise GPU investment pays for itself within 2–3 months; (4) no vendor lock-in or contract penalties; and (5) built-in GDPR compliance eliminates the need for expensive Data Processing Agreements with cloud providers. For organisations processing over 2 million LLM requests per month, the on-premise deployment is approximately 10x cheaper than any commercial alternative.

TABLE 6. SECURITY BENCHMARK COMPLIANCE

Benchmark	Coverage	Grade
OWASP LLM Top 10	100%	A+
OWASP API Security Top 10	100%	A+
NIST SP 800-218 (SSDF)	89%	A-
CIS Docker Benchmark	100%	A+
NIST AI 600-1 (AI RMF)	85%	A-

The framework was evaluated against five industry security benchmarks as shown in Table 6. The OWASP LLM Top 10 achieved 100% coverage through the three-layer guardrail architecture addressing LLM01 (prompt injection via NeMo), LLM02 (insecure output via LlamaGuard), LLM06 (PII disclosure via Presidio), LLM07 (system prompt leakage via Colang rules), LLM09 (misinformation via GroundednessChecker), and LLM10 (unbounded consumption via TokenCounter and rate limiting). The OWASP API Security Top 10 achieved 100% coverage through JWT authentication (API1: Broken Object Level Authorization), rate limiting (API4: Unrestricted Resource Consumption), RBAC enforcement (API5: Broken Function Level Authorization), and input validation across all API endpoints. NIST SP 800-218 (SSDF) achieved 89% coverage, with the remaining 11% relating to formal verification and cryptographic agility requirements that are planned for future work. CIS Docker Benchmark achieved 100% compliance through Alpine runtime images, non-root user execution, capability dropping, and read-only filesystems. NIST AI 600-1 achieved 85% coverage through hallucination detection, content safety classification, and audit logging, with the remaining 15% relating to model interpretability and fairness metrics.

VI. DISCUSSION

A. Security Properties Achieved

The framework achieves defence-in-depth across the three pillars of information security. Confidentiality is enforced through Presidio PII detection across 50+ entity types, ensuring no sensitive data passes through to or from the LLM without detection. The Presidio layer operates on both the input prompt and the LLM response, providing bidirectional PII protection that catches cases where the model might reproduce sensitive information from its training data. Integrity is maintained by the combination of LlamaGuard content safety classification across 12 harm categories (S1 through S12) and the GroundednessChecker hallucination detection using the LLM-as-Judge pattern, ensuring that harmful or factually unsupported content is blocked or flagged before reaching the end user. Availability is protected through Bucket4j rate limiting backed by Redis distributed state and Resilience4j circuit breakers with configurable failure thresholds and recovery windows. The fail-closed decision engine is the critical architectural element: because any single layer's block result denies the entire request, an attacker must simultaneously bypass all three layers to succeed—a significantly harder proposition than defeating any individual guardrail. This probability multiplication effect is the core advantage of defence-in-depth: if each layer independently catches 95% of attacks, the combined system catches 99.9875% (1 - 0.05^3), assuming independent failure modes. While real-world attack categories may exhibit correlated

failures across layers, the heterogeneous nature of our three layers (pattern matching, neural classification, and entity recognition) minimises correlation.

B. The Shadow AI Imperative

The shadow AI phenomenon presents a compelling argument for governed access over outright AI bans. Industry data shows that over 80% of employees use unapproved AI tools even when explicitly prohibited [26], with 38% sharing confidential data with unauthorised platforms [27]. Shadow AI breaches cost $670,000 more on average than traditional incidents and require 247 days to detect [28]. The Samsung ChatGPT leak [29]—where engineers disclosed proprietary code despite existing policies—demonstrates that policy-only controls fail without technical enforcement. With 77% of businesses experiencing AI-related security incidents in 2024 [28], the evidence is clear: blocking AI creates worse security outcomes than providing governed, monitored access through frameworks like SecureAI Gateway.

C. On-Premise vs Cloud Trade-offs

On-premise deployment offers clear advantages for data sovereignty: all prompts and responses remain within the organisation's network perimeter, regulatory compliance is simplified, and there are no per-request costs at scale. The data controller retains full custody of all personal data processed through the LLM system, eliminating the need for Data Processing Agreements (DPAs) with cloud AI providers and simplifying Data Protection Impact Assessments under GDPR Article 35. The on-premise model also provides predictable latency characteristics, as network round-trips to external API endpoints are eliminated. The disadvantages of on-premise deployment include infrastructure maintenance responsibility—the organisation must provision, configure, and maintain the Docker/Kubernetes infrastructure, including GPU resources for LLM inference. Model updates are the organisation's responsibility: when Meta releases LlamaGuard 4 or NVIDIA updates NeMo Guardrails, the deployment must be manually updated and regression-tested. Initial setup costs are higher than SaaS solutions, requiring hardware procurement (or allocation from existing infrastructure) and DevOps expertise for Docker Compose or Kubernetes configuration. GPU memory requirements for the LLaMA 3.1 8B model (approximately 8GB VRAM) may necessitate dedicated hardware in organisations without existing GPU infrastructure. For regulated industries—financial services (subject to PCI-DSS and banking regulations), healthcare (subject to HIPAA and HITECH), government (subject to FedRAMP and NIST 800-53), and defence (subject to ITAR and classified data handling requirements)—on-premise deployment is often the only viable option due to data residency requirements. Even for organisations in less regulated industries, the total cost of ownership analysis (Table 5) demonstrates that on-premise deployment is economically advantageous at all enterprise scales, making the data sovereignty benefits effectively cost-free.

D. Regulatory Compliance

The regulatory landscape increasingly demands the capabilities SecureAI Gateway provides. The EU AI Act (Regulation 2024/1689) [24] imposes fines of up to 35 million euros or 7% of global turnover for non-compliance, with extraterritorial scope affecting any organisation whose AI system outputs reach EU residents regardless of where the organisation is headquartered. The Act classifies AI systems by risk level and requires high-risk systems to implement risk management, data governance, transparency, human oversight, and robustness measures—all of which SecureAI Gateway's guardrail architecture, audit logging, and content safety features help satisfy. GDPR Article 25 [12] requires data protection by design and by default—a requirement directly satisfied by on-premise deployment where no personal data leaves the data controller's infrastructure. Article 35 requires Data Protection Impact Assessments for high-risk processing, and SecureAI Gateway's comprehensive audit trails provide the evidence base needed for such assessments. The right to erasure (Article 17) is simplified in an on-premise deployment because the organisation retains full control over all stored data, including chat histories and audit logs. The NIST AI Risk Management Framework (AI RMF 1.0) [25] provides voluntary guidance organised around four functions: Govern, Map, Measure, and Manage. SecureAI Gateway addresses the Measure function through its groundedness scoring (hallucination detection), token consumption tracking, and comprehensive test coverage. The Manage function is addressed through the guardrail layers, rate limiting, and fail-closed design. NIST AI 600-1 specifically identifies confabulation (hallucination) as a key risk for generative AI systems—directly addressed by the GroundednessCheckerService. For organisations subject to sector-specific regulations such as HIPAA (healthcare) or PCI-DSS (payment card industry), the on-premise deployment model eliminates the need to conduct third-party security assessments of cloud AI providers, simplifying compliance certification significantly.

VII. LIMITATIONS

1) Single LLM model tested. All experiments used LLaMA 3.1 8B served via Ollama. Results may differ with larger models (GPT-4, Claude, Gemini) or different model architectures, particularly for the LlamaGuard content safety classification layer.

2) Platform-specific evaluation. Experiments were conducted on macOS Darwin 25.4.0 with Apple Silicon (ARM architecture). Production deployments on Linux x86_64 with NVIDIA GPUs may exhibit different latency characteristics, particularly for the LlamaGuard inference layer.

3) Limited concurrency testing. Load testing reached 500 concurrent users, beyond which the 2% timeout rate suggests resource saturation. Enterprise-scale deployments (10,000+ concurrent users) have not been validated and would require horizontal scaling with Kubernetes.

4) Manual rule curation. All 50+ Colang 2.0 detection patterns were manually authored based on known attack taxonomies. No machine learning-based automatic rule generation or adversarial rule optimisation was implemented.

5) Empirical security evaluation only. No formal verification of security properties was conducted. The 100% detection rate is empirical and bounded by the test suite's

coverage of known attack patterns; novel, zero-day attack vectors may evade detection.

6) Evaluator model dependency. The GroundednessChecker's accuracy depends on the quality of the evaluator model (LLaMA 3.1 8B in our configuration). A stronger judge model would likely improve hallucination detection accuracy.

7) Approximate token counting. The TokenCounterService uses a heuristic formula (mean of chars/4 and words×1.3) rather than an exact tokenizer such as tiktoken, introducing estimation error of approximately 5–10%.

8) Single-site deployment only. The current architecture has been validated in a single Docker Compose deployment. Multi-region, high-availability configurations with Redis Cluster and database replication have not been tested.

9) Lakera Guard comparison scope. The comparative evaluation used only Lakera Guard's free tier (10,000 requests/month). The paid enterprise tier may offer improved detection rates and additional features not evaluated in this study.

10) English-only PII detection. Microsoft Presidio's pattern-based recognisers are optimised for English-language PII patterns. Detection accuracy for non-Latin scripts and multilingual documents has not been validated.

VIII. CONCLUSION AND FUTURE WORK

This paper presented SecureAI Gateway, a three-layer defence-in-depth framework for securing enterprise LLM applications. The system demonstrates that production-grade security can be achieved with minimal latency overhead (90ms guardrail processing, representing 5.6% of total request time) while maintaining zero cloud dependencies. The parallel execution architecture using Project Reactor's Mono.zip() reduces guardrail latency by 44% compared to sequential execution, making the security overhead negligible relative to LLM inference time. Beyond the core guardrail architecture, six enterprise security enhancements were implemented: Redis 7.2 distributed state for Kubernetes-ready multi-node deployments with automatic fallback to in-memory state, hallucination detection via the LLM-as-Judge pattern aligned with NIST AI 600-1, token consumption tracking for OWASP LLM10 compliance, immutable audit logs with SHA-256 request hashing for SOC 2 PI1 compliance, system prompt protection with 50+ Colang 2.0 patterns for OWASP LLM07 coverage, and automated AI red-team testing via Garak and Promptfoo integrated as Stage 14 of the Jenkins pipeline. The test suite achieves 100% JaCoCo code coverage across all six metrics (instruction, branch, line, complexity, method, and class) with 175 test methods—a result achieved through systematic branch-hunting using advanced techniques including Mockito deep stubs, Java reflection for dead-code paths, and parameterised tests for compound boolean expressions.

Comparative evaluation against Lakera Guard demonstrated superior detection rates across all categories: 100% versus 86% for jailbreak detection, 100% versus 75% for PII detection, 100% versus 92% for content safety, and 100% versus 70% for system prompt protection. These results validate the hypothesis that a multi-layer, heterogeneous guardrail architecture provides stronger protection than any single-model approach. The on-premise deployment model eliminates the data sovereignty concerns inherent in all cloud-based commercial alternatives.

The enterprise cost analysis reveals savings of 87–93% compared to commercial cloud-based alternatives across all enterprise scales, with the three-year TCO for large enterprises at approximately $140,000 versus $1.5–2 million for commercial solutions. Combined with full GDPR Article 25 [12] compliance and EU AI Act [24] readiness, SecureAI Gateway provides a compelling economic and regulatory case for on-premise LLM security. The 16-stage DevSecOps pipeline with 8 blocking security gates provides a replicable blueprint for organisations deploying LLMs in regulated environments.

Future work includes: (1) implementing DAST via OWASP ZAP integration for runtime vulnerability scanning of the REST API surface; (2) implementing horizontal pod autoscaling with Redis cluster mode for high-availability distributed state across multiple availability zones; (3) adding real-time Grafana dashboards for groundedness score trends, token consumption analytics, guardrail latency monitoring, and alert-based anomaly detection; (4) extending Kubernetes manifests with Istio service mesh for mutual TLS between sidecars, providing encrypted communication between all microservice components; (5) expanding Colang 2.0 rules for multi-language prompt injection detection covering non-English jailbreak attempts in at least 10 additional languages; and (6) implementing AI-specific monitoring for model drift detection and fairness auditing per the NIST AI Risk Management Framework, including demographic parity metrics and disparate impact analysis.

The complete source code, Docker Compose configuration, Jenkins pipeline, and comprehensive test suite are available under an open-source licence. The repository URL is withheld for double-blind review and will be provided in the camera-ready version.

REFERENCES

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in Proc. 16th ACM Workshop Artif. Intell. Secur. (AISec), 2023.

[2] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," arXiv preprint arXiv:2211.09527, 2022.

[3] H. Inan et al., "Llama Guard: LLM-based input-output safeguard for human-AI conversations," arXiv preprint arXiv:2312.06674, 2023.

[4] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in Proc. Conf. Empirical Methods Nat. Lang. Process.: System Demonstrations (EMNLP), pp. 431–445, 2023.

[5] Microsoft, "Presidio — Data protection and de-identification SDK," GitHub, 2019. [Online]. Available: https://github.com/microsoft/presidio

[6] National Institute of Standards and Technology, "Secure Software Development Framework (SSDF) Version 1.1," NIST SP 800-218, Feb. 2022.

[7] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications v1.1," 2023. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[8] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," arXiv preprint arXiv:2307.09288, 2023.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in

Proc. Adv. Neural Inf. Process. Syst. (NIPS), vol. 30, pp. 5998–6008, 2017.

[10] T. Brown et al., "Language models are few-shot learners," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 33, pp. 1877–1901, 2020.

[11] International Organization for Standardization, "ISO/IEC 27001:2022 — Information security management systems," 3rd ed., Oct. 2022.

[12] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 (General Data Protection Regulation)," Off. J. Eur. Union, vol. L 119, pp. 1–88, May 2016.

[13] VMware (Broadcom), "Spring Boot 3.x reference documentation," 2023. [Online]. Available: https://docs.spring.io/spring-boot/

[14] VMware (Broadcom), "Spring Security reference documentation," 2023. [Online]. Available: https://docs.spring.io/spring-security/

[15] Docker, Inc., "Docker documentation," 2023. [Online]. Available: https://docs.docker.com/

[16] JaCoCo Project (EclEmma), "JaCoCo — Java code coverage library," 2023. [Online]. Available: https://www.jacoco.org/jacoco/

[17] SonarSource SA, "SonarQube — Continuous inspection of code quality and security," 2023. [Online]. Available: https://www.sonarsource.com/products/sonarqube/

[18] SpotBugs Community, "SpotBugs — Find bugs in Java programs," 2023. [Online]. Available: https://spotbugs.github.io/

[19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learn. Represent. (ICLR), 2023.

[20] Cloud Native Computing Foundation, "Kubernetes documentation," 2023. [Online]. Available: https://kubernetes.io/docs/

[21] Z. Chkirbene, R. Hamila, A. Gouissem, and D. Unal, "Large Language Models (LLM) in industry: A survey of applications, challenges, and trends," in Proc. IEEE 21st Int. Conf. Smart Communities (HONET), pp. 229–234, Dec. 2024.

[22] C. L. Smith, "Understanding concepts in the defence in depth strategy," in Proc. IEEE 37th Annu. Int. Carnahan Conf. Security Technology (CCST), pp. 8–16, Oct. 2003.

[23] B. C. Das, M. H. Amini, and Y. Wu, "Security and privacy challenges of large language models: A survey," ACM Comput. Surv., vol. 57, no. 6, Art. 152, Feb. 2025.

[24] European Parliament and Council of the European Union, "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)," OJ L, 2024/1689, 12 Jul. 2024.

[25] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, Jan. 2023.

[26] Cybersecurity Dive, "Shadow AI is widespread — and executives use it the most," Nov. 2025.

[27] A. Patel, "AI gone wild: Why shadow AI is your IT team's worst nightmare," Cloud Security Alliance Blog, Mar. 2025.

[28] Reco, "AI and cloud security breaches 2025: Trends, statistics, and lessons learned," 2025.

[29] Lasso Security, "LLM data privacy: Protecting enterprise data in the world of AI," 2025. [Online]. Available: https://www.lasso.security/blog/llm-data-privacy

[30] Menlo Ventures, "2025: The state of generative AI in the enterprise," 2025.

[31] Ataccama (reported by Datacentre Solutions), "72% of data leaders fear failure to adopt AI will result in competitive disadvantage," 2025.

[32] Grand View Research, "Artificial intelligence in cybersecurity market report," 2025.

[33] SiliconANGLE, "F5 acquires AI security provider CalypsoAI for $180M," Sept. 2025.

[34] Palo Alto Networks Unit 42, "Comparing LLM guardrails across GenAI platforms," 2025. [Online]. Available: https://unit42.paloaltonetworks.com/comparing-llm-guardrails-across-genai-platforms/

[35] R. J. Young, "Evaluating the robustness of large language model safety guardrails against adversarial attacks," arXiv preprint arXiv:2511.22047, 2025.

[36] F. Yin, P. Laban, X. Peng, Y. Zhou, Y. Mao, V. Vats, L. Ross, D. Agarwal, C. Xiong, and C.-S. Wu, "BingoGuard: LLM content moderation tools with risk levels," in Proc. Int. Conf. Learn. Represent. (ICLR), 2025.

[37] Lakera AI, "Lakera Guard: AI security for LLM applications," 2025. [Online]. Available: https://www.lakera.ai/lakera-guard

[38] Amazon Web Services, "Amazon Bedrock Guardrails pricing," 2025. [Online]. Available: https://aws.amazon.com/bedrock/pricing/

[39] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications v2.0," 2025. [Online]. Available: https://genai.owasp.org/

[40] Y. Tang, Y. Liu, J. Lan, Z. Yan, and E. Gelenbe, "Security of LLM-based agents regarding attacks, defenses, and applications: A comprehensive survey," Information Fusion, vol. 127, 2026.